

# Project Plan

Computer Games – Phase B

Topic: Path Planning

**Group:** 1      **Members:** Franziska Plagwitz, Luna, Sem Massa

## Overview

This document defines our shared approach for the project in the Computer Games Exercises, Phase B. Our assigned topic is **Path Planning**. The goal is a small game or demo application that demonstrates at least three techniques covered in the lecture and additionally includes one recently developed technique. The focus lies on a direct, measurable comparison of different path planning methods.

## 1 Big Picture Goal

We will build a **pursuit demo** in **Godot**: a pursued actor (either player controlled or with simple flee logic) moves through a scene containing static and dynamic obstacles. Two to three AI controlled pursuers navigate in parallel using different path planning algorithms toward the same target. During the simulation, benchmarks are recorded live and displayed as a comparison chart at the end.

### 1.1 Game Concept in Detail

- **Scenario:** top down view, a grid or navigation structure with obstacles (walls, rooms, narrow passages), similar to the example scenes from the lecture (waypoints, grid, polygon meshes).
- **Mode 1, Spectator Race:** multiple agents start at the same point and pursue the same moving target. Whoever arrives first, or follows the shortest, fastest route, wins visibly.
- **Mode 2, Live Benchmark:** on button press, a course is solved by several algorithms simultaneously, without a player character, purely for data collection.
- **Dynamic obstacles:** some obstacles move on their own so that replanning becomes necessary. This makes the difference between algorithms visible and provides material for the recent techniques.

## 1.2 Proposed Techniques (Lecture Integration)

The following points outline various techniques and lecture concepts that could be integrated into the project. These serve as suggestions and options for implementation rather than rigid specifications:

1. **Path Planning & Navigation (Core Topic):** A direct comparison of algorithms (e.g., Dijkstra/BFS, Greedy Best-First, and  $A^*$ ) could be conducted. For this, different discrete navigation abstractions mentioned in the lecture might be utilized, such as a uniform raster/grid, waypoint graphs, or polygon meshes (NavigationMesh).
2. **Game Engine Architecture & Messaging (Potential Approach):**
  - *Messaging System:* To reduce system coupling instead of hardcoding direct dependencies, the live benchmark could utilize Godot's signal architecture as an asynchronous, event-driven message bus. Agents might emit detached data objects caught by a central **BenchmarkManager**.
  - *State Machines:* Finite state machines (FSM) represent a viable option to handle behavioral switches for the pursued actor and to track replanning phases.
3. **Collision Detection Phases & Proxies (Optional Optimization):**
  - *Broad Phase vs. Narrow Phase:* The optimization split could be explored by relying on the engine's broad-phase spatial partition hierarchies (Grid/BVH) to eliminate non-colliding pairs, while precise narrow-phase geometric tests handle near-field agent-to-obstacle resolution.
  - *Bounding Boxes & Proxies:* Incorporating Axis-Aligned Bounding Boxes (AABB) or simple bounding circles as geometric proxies is another possibility. Dynamic obstacles changing their spatial occupancy could then trigger localized graph updates and instant path replanning.
4. **Graphics Pipeline & Transformation Spaces (Suggested Concepts):**
  - *Transform Spaces & Model Matrices:* Agent coordinates could be mapped through distinct affine spaces. For instance, every agent might utilize localized 2D translation and rotation transformation matrices to project coordinate points into the shared Global/World Space.
  - *Scene Tree Hierarchy:* Godot's hierarchical tree node structure could be used to demonstrate parent-child relative transform propagation (e.g., obstacles attached to a moving compound platform inheriting spatial translation).
5. **Particle Simulations (Optional Visual Feature):**
  - To visibly differentiate competing path planning strategies, agents could optionally emit real-time, color-coded trailing paths utilizing particle emitters. This simulation might map to the lecture's four-step particle life cycle mechanics: 1) *Produce/Inject* at current spatial nodes, 2) assigning individual *Attributes* (initial reverse velocity, color tags), 3) scripted *Transport/Modification*, and 4) deletion once time-to-live (*TTL*) limits *Vanish*.

## 2 Work Packages

The three work packages are scoped so they can be tackled largely in parallel.

### WP1, Game Foundation, Scene, and Collision Detection

**Responsible:** \_\_\_\_\_

**Goal:** a playable Godot scene with obstacles, a navigation structure, and working collision detection, on top of which the algorithms from WP2 will operate.

**Tasks:**

- set up the Godot project, build a scene with static obstacles
- choose and implement a navigation structure (grid, waypoint graph, or NavigationMesh, decided jointly with WP2 since this forms the basis for the algorithm comparison)
- collision detection: queries between agent and wall, and between agents themselves
- optional: dynamic obstacles with their own movement
- player character or flee logic for the target
- basic movement and steering of agents along a given path (path following)

**Interface to WP2:** provides the graph structure (nodes, edges, weights) on which the pathfinding algorithms operate.

### WP2, Path Planning Algorithms and Benchmarks

**Responsible:** \_\_\_\_\_

**Goal:** implementation of the algorithms to be compared, together with a benchmark framework that records metrics during the simulation.

**Tasks:**

- literature research on recent papers for the recent technique, and selection of benchmark metrics (e.g. path length, number of expanded nodes, computation time, success rate under dynamic obstacles, replanning frequency)
- implement dynamic programming / breadth first search as baseline
- implement greedy best first search
- implement A\* search
- implement the chosen recent technique (e.g. Jump Point Search)
- benchmark framework: logging of metrics per algorithm and run
- visualization of paths in the scene (e.g. color coded per algorithm)

**Interface to WP1:** requires the graph structure from WP1 as input, returns computed paths back to agent steering.

**Interface to WP3:** provides the raw benchmark data for evaluation and the report.

### WP3, Evaluation, Report, and Presentation

**Responsible:** \_\_\_\_\_

**Goal:** a clearly structured report with proper references that explains the techniques and evaluates the benchmark results, plus the final presentation.

**Tasks:**

- structure of the report (outline, organization, LaTeX template)
- evaluation of the benchmark data from WP2: charts, tables, statistical assessment
- comparison with results from recent papers (assessing whether our results align with the literature)
- theory section: explanation of the techniques used, with references
- merging contributions from all three work packages into a consistent report
- preparation of the presentation (slides, live demo of the game)
- rehearsal presentation

**Note:** this work package starts somewhat later in time, since it depends on initial results from WP1 and WP2, but runs in parallel content wise (e.g. the theory section can already be started early).

### 3 Open Decisions

- final choice of the recent technique (suggestion: Jump Point Search, since it compares well against A\* and is well documented in recent literature)
- navigation structure: grid versus waypoint graph versus NavigationMesh (Godot provides NavigationMesh natively, which eases implementation but may offer less control over the graph structure for the algorithm comparison)
- scope of dynamic obstacles (purely static would be simpler, dynamic makes the comparison more interesting and demonstrates replanning)